✦ paldora ✦

a box for your people.



Product concept, architecture, security, roadmap, and business model

Developer alignment document • v0.1 • August 2026

✦ **PRIVATE • TIMELESS • YOURS** ✦

Executive summary

Paldora is a private, expressive social space for people who genuinely know each other. It combines the personal identity of MySpace, the flexible posting culture of Tumblr, the visual collecting of Pinterest, and a privacy-first philosophy—without becoming another public engagement feed.

The initial use case is deliberately small: a friend group of eight sharing life updates, photos, thoughts, music, trips, and milestones. The architecture should nevertheless treat groups, permissions, storage, and entitlements as first-class concepts so the product can later support many independent circles.

> *Paldora should become more valuable with time. It is not optimized for infinite scrolling; it is optimized for remembering.*

Principle	Meaning
Close friends first	Optimize for meaningful relationships rather than follower counts.
Private by default	Users should always understand who can see a post.
Expression over uniformity	Profiles, themes, post types, music and visual identity should feel personal.
Chronology over manipulation	Start with a predictable chronological feed, not engagement-maximizing ranking.
Memories compound	Timelines, recaps, archives and yearbooks turn old content into value.
Portable architecture	Web first, mobile later, shared TypeScript domain logic and an evolvable backend.

One-sentence pitch

Paldora is a living scrapbook for close friends: a private place to share what is happening in your life and keep the story of your friendships over time.

V1 non-goals

- Replacing WhatsApp or Signal as a full messaging product.
- Building a public influencer economy.
- Algorithmic recommendations or viral discovery.
- Direct video hosting at launch.
- Inventing a custom cryptographic protocol.

- Building microservices before scale requires them.

Problem, audience & positioning

Problem

Close friends increasingly live in different cities, universities, jobs, and time zones. Group chats are excellent for conversation but poor as a durable archive. Public social platforms encourage broadcasting, performance, recommendations, and short-lived content. Shared photo libraries preserve images but not the context around a life.

Primary audience

The first audience is an existing real-world friend group that wants a low-pressure way to stay current without constant synchronous conversation. Natural expansion segments include university friend groups, long-distance friends, families, travel groups, and other small trusted circles.

Positioning

Category	Paldora stance
Group chat	Complement it; preserve updates that should still matter months later.
Instagram / TikTok	Less public and performative; no follower race or entertainment-first feed.
Shared albums	Add narrative, identity, comments, life updates, themes, and chronology.
Private social apps	Differentiate through expressive profiles + long-term archive + privacy.
Signal	Privacy inspiration only; never claim equivalent security without audited E2EE.

Brand

The working name Paldora comes from “pal” + “Pandora.” The metaphor is a box containing the evolving stories, photos, memories, and surprises of the people you care about. Use the box metaphor lightly.

Working lines: “a box for your people.” • “your box, your people.” • “open it. share it. keep it.”

Core experience

Core loop

Open Paldora → see meaningful updates from your people → react/comment → share something from your own life → return for new updates or resurfaced memories.

Surface	Purpose	V1
Home feed	Chronological posts from permitted people/circles.	Yes
Profile / room	Identity, posts, photos, bio, theme, music.	Yes
Composer	Text, photos, links, quote/note, structured Life Update.	Yes
Life Updates	Periodic structured check-ins that make catching up easy.	Yes
Memories	Timeline/archive of posts and photos.	Yes
On This Day	Resurface past moments.	Later
Boards / collections	Pinterest-like saving and organizing.	Later
Messaging	Realtime private chat.	No
Public discovery	Content beyond existing relationships.	No

Post model

Use a common post envelope plus typed/versioned content. Initial types: text, photo/gallery, quote/note, link, and life_update. Later: music, poll, location/trip, milestone, video.

Life Update

A Life Update is Paldora's signature format: a low-pressure monthly or ad-hoc check-in with optional sections such as life, work/study, travel, currently into, photos, and what is next. A group overview can show recent updates without shaming people who have not posted.

✦ 04 / DESIGN LANGUAGE

Visual & interaction direction

The UI should feel like a carefully made digital scrapbook rather than a generic SaaS dashboard: warm paper, dark ink, pink/lavender accents, taped photos, polaroid framing, handwritten details, and intentionally imperfect decoration. Functional UI remains crisp and accessible beneath the aesthetic layer.

Brand palette: Obsidian Ink • Antique Gold • Champagne • Warm Ivory • Soft Lavender. Use gold for key accents, dividers, premium states, badges and tiny star/sparkle motifs—not as a full-page fill.

Source	Borrow	Avoid
MySpace	Personal rooms, themes, music, Top Friends energy.	Arbitrary HTML/CSS or unsafe customization.
Tumblr	Flexible post types, expressive writing, chronological culture.	Complex reblog graphs in V1.
Pinterest	Masonry galleries, collections, visual browsing.	Recommendation-first product design.
Scrapbooks	Annotations, chronology, tactile memory value.	Decoration that harms readability/performance.

Customization

Never allow arbitrary user HTML or JavaScript. Store a validated theme configuration and render safe components. Theme data may include preset, accent, background asset, typography preset, widget order, profile music reference, banner, and decorative stickers.

Accessibility baseline

- Strong contrast for functional text and controls.
- Keyboard-accessible web navigation and forms.
- Reduced-motion support.
- Alt text for meaningful images; decorative assets ignored by assistive tech.
- Never communicate privacy/status by color alone.
- Responsive mobile web before the native app exists.

✦ 04A / LIGHT MODE & PERSONALIZATION

Theme system and personal rooms

Paldora must support a complete light mode alongside the premium dark experience. Light mode should feel intentional—not like an inverted dark theme. Use warm ivory and champagne surfaces, antique-gold accents, soft shadows, and dark readable typography.

Appearance modes

Every first-party screen should support Light, Dark, and System/Auto appearance. Theme personalization is a separate layer: appearance controls the application chrome, while a user's Room theme controls their expressive profile space.

MySpace-inspired personalization, safely constrained

Users should be able to make their Paldora Room unmistakably theirs without executing arbitrary code. Personalization can include curated theme presets, custom accent colors, approved font families, background textures/images, banners, profile music references, decorative stickers/sparkles, widget visibility, widget ordering, gallery layout, and optional Top Friends-style modules.

A validated theme schema should be versioned so old profiles continue rendering after the design system evolves. Accessibility overrides must win where necessary: minimum contrast, readable font sizing, reduced motion, and safe fallback colors.

Theme presets

Launch with a small set of polished presets—e.g. Obsidian Gold, Cream Paper, Lavender Dream, Ocean Glow, Daisy Day, and Neon Night—then let users personalize within controlled tokens. Premium can later expand the theme library without making basic self-expression paywalled.



Concept direction: light/dark application modes plus MySpace-inspired personal Room themes.

Implementation boundary

Do not store raw CSS, HTML, or JavaScript from users. Store structured theme data and render it through trusted components. Uploaded backgrounds and decorative media use the same private-storage, validation, moderation, and quota rules as other user media.

Recommended stack

Layer	Choice	Rationale
Language	TypeScript	Shared language across web, domain packages and future mobile.
Web	Next.js + React	Strong web UX, routing, server rendering and ecosystem.
Styling	Tailwind CSS + tokens	Fast iteration with coherent visual rules.
Mobile	Expo + React Native	Native iOS/Android later with shared TypeScript logic.
Backend	Supabase initially	PostgreSQL, Auth, Storage and Realtime for a fast robust MVP.
Database	PostgreSQL	Relational integrity, indexing, transactions and long-term scalability.
Validation	Zod	Shared runtime validation and typed contracts.
Repo	pnpm + Turborepo	Monorepo for web/mobile/shared packages.
Testing	Vitest + Playwright	Domain/unit tests plus critical browser journeys.

Repository shape

```
apps/
  web/              Next.js
  mobile/           Expo (later)

packages/
  domain/           types, rules, entitlements
  api/              service functions
  validation/       Zod schemas
  ui-tokens/        shared design tokens
  config/           lint / TypeScript config

supabase/
  migrations/
  seed/
  functions/        only where needed
```

React components should call domain/API services rather than scattering raw Supabase queries throughout the UI. This keeps authorization reviewable and makes a later dedicated API possible without rewriting every screen.

Core entities

Entity	Purpose / important fields
profiles	user_id, username, display_name, bio, avatar, banner, theme_config
boxes	id, name, owner_id, privacy_mode, created_at
box_members	box_id, user_id, role, joined_at, status
friendships / follows	Relationship graph if expansion goes beyond Box-only membership.
posts	id, author_id, box_id, type, visibility, created_at, updated_at
post_payloads	Typed or versioned JSON content; never unsafe arbitrary markup.
post_media	post_id, storage_path, media_type, dimensions, position
comments	post_id, author_id, parent_comment_id, payload, created_at
reactions	user_id, post_id, reaction_type
notifications	recipient, actor, type, entity, read_at, created_at
invites	box_id, email/token hash, inviter, expiry, accepted_at
subscriptions	box_id, provider IDs, plan, status, current_period_end
storage_usage	box_id, bytes_used, recalculated_at

Database rules

- Unique normalized usernames.
- Unique membership per (box_id, user_id).
- Explicit foreign keys and deletion behavior.
- Reaction uniqueness enforced in the database.
- Consistent generated timestamps.
- RLS-backed visibility and membership authorization.
- Schema changes through migrations only.

Pagination

All feeds, comments, notifications, galleries, and search results are paginated from V1. Prefer cursor/keyset pagination for time-ordered feeds instead of large OFFSET queries.

Security model

V1 should be a strongly secured private social application, not marketed as Signal-equivalent E2EE. Security is a product requirement, not a cleanup phase.

Control	V1 expectation
Authentication	Verified account flow; secure sessions; optional MFA/passkeys as supported.
Authorization	RLS on exposed tables plus server-side checks for privileged operations.
Invites	Invite-only beta; short-lived/single-use tokens; hash secrets where applicable.
Transport	HTTPS only; HSTS where production setup supports it.
Storage	Private buckets and authorized/signed access; URLs are not access control.
User content	No arbitrary HTML/JS; sanitize rich text; validate MIME, size and dimensions.
Browser defenses	CSP, XSS prevention, secure cookie patterns, CSRF-aware mutations.
Abuse	Rate-limit auth, invites, uploads, comments, reactions and expensive endpoints.
Secrets	No service-role keys in clients; secrets remain server-side.
Operations	Security-sensitive audit events, dependency updates, backups and restore tests.

Future E2EE

A future encrypted Box could encrypt post bodies, comments and media on the client before upload. This requires mature device/group key management, membership-change handling, recovery, revocation and new-device enrollment. Do not invent a protocol. Use established cryptographic designs/libraries and obtain independent review before strong marketing claims.

Keep metadata separate from versioned content payloads now so encrypted payloads can be introduced later without replacing the entire schema.

From eight friends to a platform

Do not build for millions now. Build clean boundaries so proven bottlenecks can be replaced when evidence justifies it.

Stage	Focus
8–100	Next.js + Supabase; correctness, UX, RLS, backups, image compression.
100–5k	Indexes, query plans, pagination, media optimization, observability, rate limits.
5k–50k	Dedicated API where justified, queues/workers, CDN strategy, moderation tooling.
50k+	Cache hot paths, feed fan-out strategy, search service, async notifications, cost controls.
Large scale	Split services only around measured bottlenecks; keep PostgreSQL until workload says otherwise.

Likely first bottlenecks

- Photo/video storage and bandwidth.
- Feed query patterns and indexes.
- Notification fan-out.
- Search.
- Image transformation and thumbnails.
- Abuse/moderation workload.
- Operational visibility and cost attribution.

Background jobs

Image processing, notifications, exports, recaps, yearbook generation and deletion workflows should move to queue/worker execution as usage grows.

Premium without compromising trust

Preferred model: subscriptions and optional physical products rather than targeted advertising. Working principle: users pay Paldora, so advertisers do not.

Subscription unit

The Box/circle should own the subscription. One person can pay and the group benefits, reducing friction compared with asking every friend to subscribe.

Free	Paldora+ (illustrative)
Core posting, comments, reactions	Much larger shared storage
Profiles and basic themes	Original-quality media / video
Private circles	Advanced themes/customization
Basic memory archive	Advanced recaps, exports and backup options
No targeted ads	Potential premium privacy features
Enough storage to experience value	Physical yearbooks/prints can remain separate purchases

Entitlements

Every premium-sensitive capability should query a centralized entitlement service. Enforcement happens server-side. Hiding a premium button is UX, not security.

Downgrade rule: never immediately delete memories when a subscription expires. Preserve existing content, disable new uploads above quota, and provide upgrade/export/delete options.

Potential revenue

- Box subscription (monthly/annual).
- Storage tiers.
- Video/original-quality media.
- Premium themes/customization.
- Printed yearly memory books and photo products.
- Potential family/larger-circle plans.

Delivery plan

Milestone	Scope	Exit criteria
M0 Foundation	Repo, environments, migrations, auth, RLS skeleton, CI.	Clone/run/test/deploy safely.
M1 Private beta	Invites, profiles, feed, text/photo posts, comments, reactions.	Original group can use it daily.
M2 Identity	Themes, profile rooms, music references, Life Updates, memories.	Feels unlike generic social CRUD.
M3 Reliability	Image pipeline, quotas, notifications, exports, moderation basics, monitoring.	Safe to invite outside groups.
M4 Mobile	Expo app, push, camera/photo workflow, shared domain package.	Core flows are native-quality.
M5 Growth	Multiple Boxes, admin tools, billing entitlements; discovery only if justified.	Independent groups can onboard.
M6 Privacy+	Evaluate client-side encryption/E2EE and independent review.	Claims match implementation.

Effort expectations

AI-assisted development can make a functional web MVP fast, but a version safe to leave running for real users should be budgeted in weeks rather than days. Security review, polish, mobile distribution, billing and E2EE are separate complexity multipliers.

Team working practices

- Main is always deployable; keep feature branches short.
- Every schema change is a reviewed migration.
- No production secrets or service-role credentials in client code or Git.
- Authorization tests accompany new private data paths.
- Critical Playwright flows: sign in, invite, create post, visibility, delete, upload.
- Domain rules live outside presentation components.
- Use feature flags for incomplete/experimental capabilities.
- Log useful request/context IDs without unnecessarily logging private content.
- Never copy production user data into local development.
- Test backup restore, not only backup creation.
- Set performance budgets for media payloads and unbounded lists.
- Accessibility and responsive behavior are acceptance criteria.

Definition of done

Area	Question
Product	Does behavior match the user story, including empty/loading/error states?
Security	Who can read/perform it, and is that enforced server/database-side?
Data	Are constraints, migration, deletion behavior and pagination defined?
Quality	Are tests appropriate to the risk?
UX	Does it work on mobile web, keyboard, slow network and failure states?
Operations	Can failures be observed and changes rolled back safely?

Core user stories

Area	Story
Account	As an invited person, I can create an account and join the intended Box without exposing it to outsiders.
Profile	I can create a personal room and safely customize its visual identity.
Feed	I see a chronological, paginated feed of posts I am authorized to read.
Post	I can create, edit and delete my own text/photo posts.
Media	I can upload compressed images; unauthorized users cannot fetch private originals.
Interaction	I can comment/react, and authors receive an in-app notification.
Life Update	I can publish a structured update that friends can scan quickly.
Privacy	Before publishing, I can clearly see the audience for the post.
Memory	I can browse my Box chronologically by month/year.
Exit	I can export my content and request account deletion with predictable consequences.

Open product questions

- Is membership primarily Box-based, friend-based, or both?
- Can one user belong to multiple Boxes in the first public release?
- What profile data is visible across Boxes?
- Single heart or multiple reaction types?
- How should music integration work legally and technically?
- What free storage quota fits actual usage/cost?
- Should Life Updates be periodic prompts or entirely voluntary?
- What is the long-term data portability/export promise?
- What happens to shared content when a Box owner leaves?

Data ownership & deletion

A memory product earns trust by making data lifecycle behavior predictable. Deletion, export, retention, backups, shared ownership and account closure need explicit rules before public launch.

Scenario	Desired behavior
Delete own post	Remove it from normal access promptly; propagate to media and derived artifacts.
Delete account	Explain impact on authored posts, comments and shared Box history before confirmation.
Leave Box	Remove future access; define whether authored shared content remains or is withdrawn.
Delete Box	Owner/admin flow with strong confirmation, export opportunity and grace period where appropriate.
Export	Machine-readable metadata plus user media in a documented archive format.
Backup retention	Document operational retention; ensure expired backups age out according to policy.
Premium lapse	Do not delete over-quota memories immediately; freeze new uploads until resolved.

Moderation for public growth

The original eight-person beta needs little community moderation. Public growth changes that. Before open signup, implement blocking, reporting, account suspension, abuse rate limits, admin review tools, legal/privacy documents, and a clear escalation path.

Measure connection, not addiction

Avoid optimizing for time-on-app as the primary success metric. Paldora should measure whether groups form, return, share meaningful updates, and build a durable archive.

Metric	Why it matters
Activated Box	A Box with multiple accepted members and at least one meaningful post.
Group retention	Whether the same Box remains active after 4/8/12 weeks.
Contributor breadth	How many members post, not just whether one power user posts often.
Life Update completion	Whether the signature format creates useful catch-up behavior.
Memory revisit rate	Whether old content becomes valuable again.
Invite conversion	Whether members can successfully bring their real friends in.
Storage growth / active Box	Critical for cost and future premium design.
Export/delete success	Trust metric: users can leave cleanly when they choose.

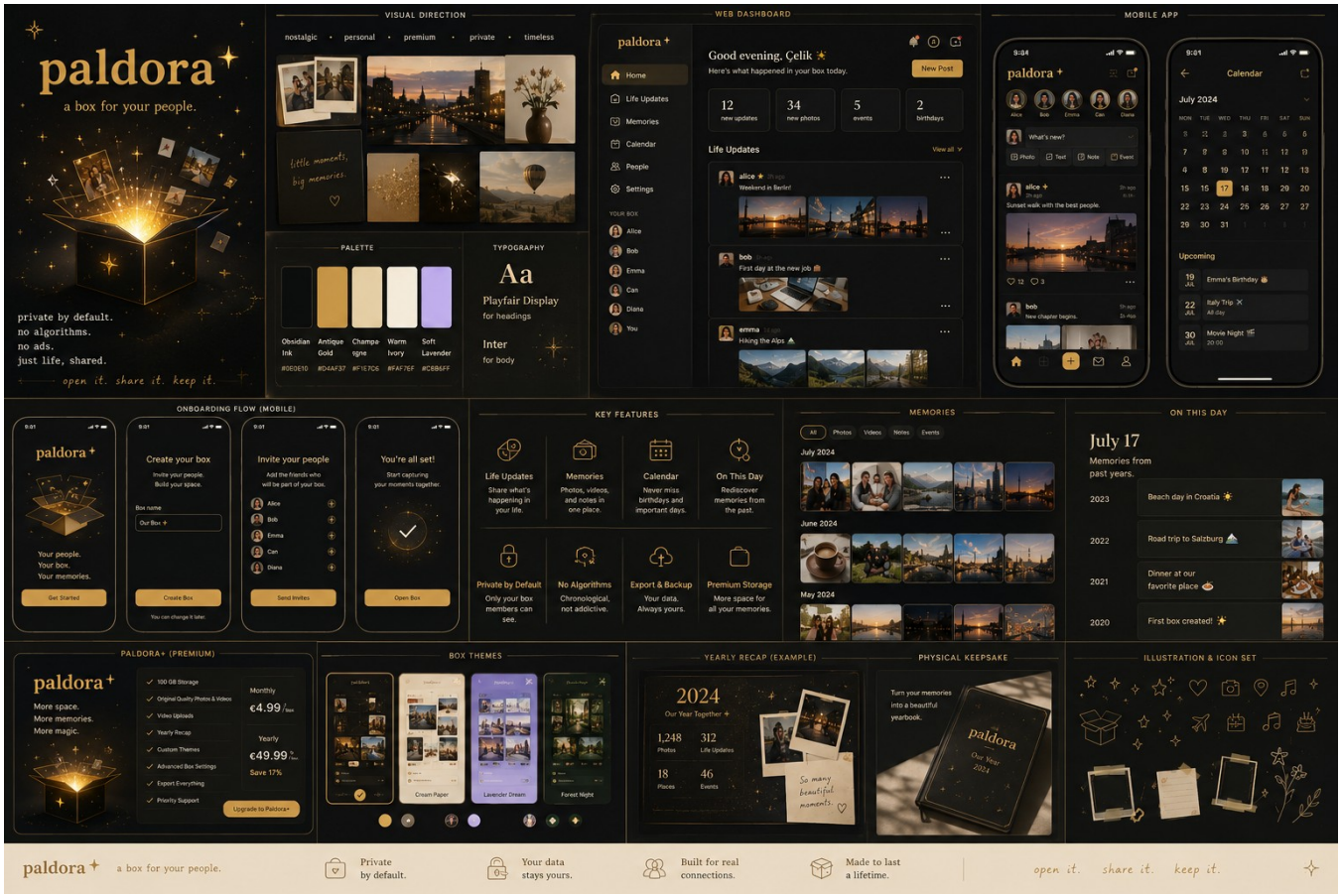
Do not collect analytics merely because they are available. Minimize event payloads, avoid sensitive post content, and document what is collected.

Immediate decisions for the development team

- Use TypeScript end-to-end for application code.
- Build web first with Next.js; mobile comes later with Expo.
- Use PostgreSQL/Supabase for the first production architecture.
- Treat Box as a first-class entity even for the original eight-person group.
- Make visibility and authorization explicit in schema and RLS.
- Centralize data access and premium entitlements behind service modules.
- Compress/resize images from V1; do not store image blobs in PostgreSQL.
- Keep direct video uploads out of V1.
- Do not allow arbitrary HTML/JavaScript profile customization.
- Do not claim E2EE/Signal-level security until it truly exists and is independently reviewed.
- Start invite-only and chronological.
- Optimize the product around long-term memories, not public follower growth.

Updated visual reference

The current premium visual direction uses antique gold and champagne as brand accents, with sparkles as a restrained signature motif. The concept board below should guide brand-level decisions; individual Room themes are intentionally allowed to diverge.



Paldora premium brand system and product concept board.

Suggested first sprint

Day / workstream	Deliverable
Foundation	Monorepo, lint/typecheck/test, local Supabase, env conventions, CI.
Identity	Auth, invite flow, profile schema, username normalization.
Boxes	Box + membership schema, roles, RLS and seed data for the first group.
Posting	Post schema, composer, feed, edit/delete, pagination.
Media	Private storage, client compression, upload validation, thumbnails.
Interaction	Comments, reactions, notification table and basic UI.
Hardening	Authorization tests, error states, backup plan, deployment checklist.

Closing principle

Build the smallest version that the original group genuinely wants to keep using, while preserving the architectural seams needed for multiple circles, premium, mobile, and stronger privacy later.

HTML/CSS personalization across web and mobile

Paldora should preserve the creative freedom that made early MySpace profiles memorable, while keeping authentication, navigation, payments, privacy controls, and other trusted application surfaces outside user-controlled markup. The goal is expressive personal Rooms, not arbitrary code execution.

Personalization tiers

- Simple themes — presets, Light/Dark/System appearance, accent colors, approved fonts, backgrounds, banners, stickers, sparkles, widget order, gallery layout, and profile music references.
- Advanced styling — scoped custom CSS that can style only the user's Room surface and approved components.
- Custom HTML widgets — sanitized HTML rendered in isolated containers. These can be placed inside a Room but cannot replace trusted Paldora navigation or security UI.

Security boundary

Arbitrary JavaScript is not allowed. HTML is sanitized with an allowlist, dangerous event attributes are removed, external embeds are restricted or allowlisted, and custom CSS is scoped to the Room. Customization must never be able to imitate or cover login, account security, privacy, billing, moderation, or navigation interfaces.

- Allowed HTML examples: div, span, p, headings, lists, blockquote, strong/emphasis, approved links and images.
- Disallowed or tightly restricted: script, object, embed, forms/inputs, base/meta, inline event handlers, arbitrary remote JavaScript.
- Prefer Paldora-hosted uploaded backgrounds/assets over arbitrary remote asset URLs to reduce tracking and privacy leakage.
- Apply Content Security Policy, sanitizer tests, size limits, rate limits, media validation, and abuse reporting to customizable content.

Web implementation

On the web, standard themes render through trusted React components. Advanced CSS is transformed/scoped to a Room root. Custom HTML widgets should render inside a strongly isolated sandbox (for example a sandboxed iframe or equivalent boundary) with no access to Paldora session state. The exact sandbox flags and CSP must be security-reviewed before release.

Mobile implementation

The native Expo/React Native application should not turn the entire profile into a WebView. The Paldora shell, navigation, profile identity, privacy indicators, post actions, settings, and account controls remain native. Standard themes are rendered with native components from the same structured theme configuration used by the web client.

Advanced HTML widgets may be rendered inside isolated WebView regions within the Room. The WebView receives only the minimum data required to render that widget, has navigation and bridge capabilities locked down, and cannot access authentication tokens or invoke privileged native actions. This preserves native performance and accessibility while still allowing MySpace-like creativity.

Shared theme contract

Web and mobile share the theme schema, not necessarily the rendering engine. A versioned ThemeDefinition should describe colors, typography tokens, background assets, decorations, widget layout, visibility, and capabilities. Web maps those tokens to CSS/React; mobile maps them to React Native styles. HTML-widget content is an optional advanced capability layered on top.

Theme editor UX

- Simple / Advanced mode toggle.
- Live preview for desktop and phone breakpoints.
- Draft themes that are not visible until published.
- Reset to safe default at any time.
- Accessibility warnings for poor contrast or unreadable combinations.
- Import/export of a safe Paldora theme format can be considered later.

Theme sharing and marketplace

Longer term, users can publish reusable themes without exposing executable code. A theme marketplace/community library could include Y2K Glitter, Dark Academia, Frutiger Aero, Tumblr-era, Cream Paper, Obsidian Gold, and other styles. Theme packages should contain validated tokens/assets and optionally reviewed HTML widget templates. Basic self-expression should remain available on the free tier; premium can expand advanced layouts, storage-heavy assets, animated decorations, and curated theme packs.

Engineering acceptance criteria

- A malicious Room cannot read another user's data or session credentials.
- A custom theme cannot modify trusted Paldora UI outside its Room boundary.
- A Room renders acceptably in Light and Dark application shells.
- Standard themes render natively on iOS/Android without requiring a full-page WebView.
- Custom widgets fail closed: if sanitization or sandboxing fails, the widget does not render.

- Theme schema is versioned and migrations/fallbacks preserve old profiles.
- Reduced-motion and accessibility overrides take precedence over decorative effects.

Design principle: Paldora should let profiles look wildly different while the security model remains boring and predictable.

Paldora visual system — consolidated reference

This updated visual reference consolidates the current Paldora direction: premium black-and-antique-gold branding, a full warm light mode, responsive web and native mobile feed concepts, expressive Room presets, web and mobile personalization editors, privacy-first feature language, and the proposed platform architecture.

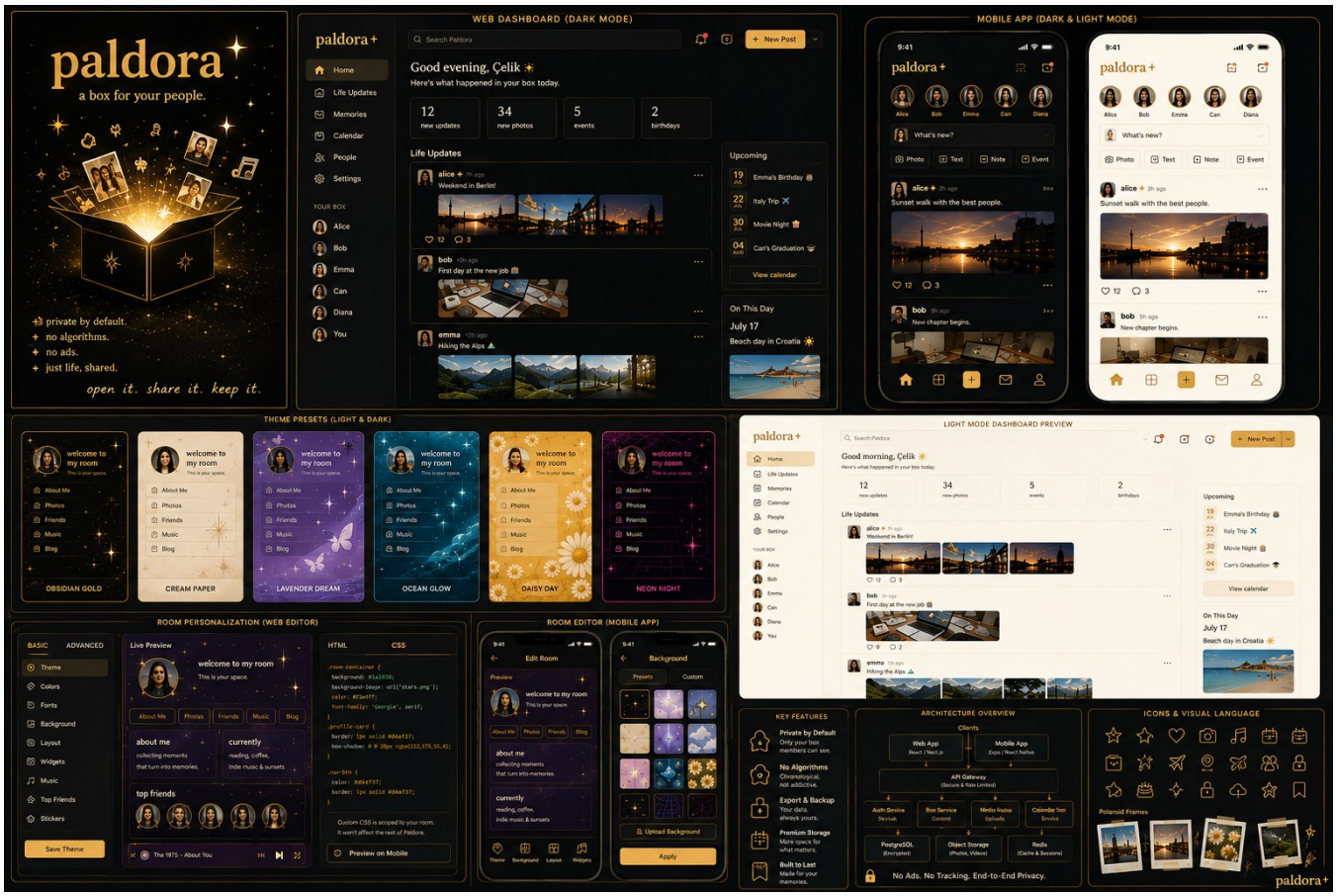


Figure: consolidated Paldora UI and brand concept — web, mobile, themes, personalization, features and architecture.

Visual decisions represented

- Dark mode uses near-black/obsidian surfaces with antique-gold hierarchy rather than pure black/yellow contrast.
- Light mode uses warm ivory and cream surfaces with restrained gold accents so it feels premium and intentional.
- Mobile retains the same brand identity while using native navigation patterns and compact post composition.
- Room themes are allowed to diverge dramatically from the core application shell, preserving MySpace-like individuality.

- Sparkles, stars and subtle magical-box imagery are signature motifs, but should be used as accents rather than constant decoration.
- The visual system supports both polished default themes and advanced user personalization without weakening trusted application UI.

Design handoff note

Concept art is directional rather than a pixel-perfect implementation specification. Production UI should be rebuilt from design tokens and reusable components, then validated for accessibility, responsive behavior, localization, performance, privacy-state clarity and native mobile conventions.